



Powered by
Arizona State University®

University of Donja Gorica

Faculty of Information Systems and Technologies

Podgorica

Developing a 3D game using Unity

THESIS

Mentor: prof. Dr Tomo Popović

Co-mentor: mr Stevan Čakić

Student: Andrija Radunović

Index number: 21/016

ABSTRACT

This thesis presents the design, development, and evaluation of a playable prototype for a Soulslike action role-playing game (RPG) using the Unity game engine. The project aims to explore how Unity's broad toolset and scripting environment can support the creation of complex and genre-specific gameplay systems such as deliberate combat, difficulty, action rpg elements and immersive feedback while working within the constraints of solo development. Drawing inspiration from titles like Elden Ring and Dark Souls, the prototype focuses on responsive player mechanics, networked multiplayer foundations, and a system architecture tailored to rapid iteration and testing. The methodology combines agile and iterative development practices with documentation, using Unity's component-based framework and C# scripting to implement core features including movement, animation, combat, UI, and persistent game state. Throughout the process, technical and design challenges, such as hit detection accuracy, animation coordination, and system integration, were addressed through continuous testing, debugging, and reflection. Results demonstrate the successful implementation of key Soulslike mechanics, with stable performance, responsive controls, and a functional networked multiplayer foundation. However, the project's scope and limitations, such as a restricted playable environment and incomplete feature set, highlight both the strengths and challenges of solo development. The thesis concludes with an evaluation of the prototype and recommendations for future improvements. It highlights Unity's effectiveness as a platform for prototyping gameplay systems and offers valuable insights for aspiring game developers working in similar genres.

CONTENT

ABSTRACT	2
1. Introduction	5
1.1 Objectives and scope	6
1.2 Sources of Data and Methods of Collection	7
1.3 Content and Structure of the Thesis	8
2. Background	9
2.1 Game development principles for Action RPGs	10
2.2 Characteristics of the “Souls-like” genre	11
2.4 Overview of game engines	12
2.5 C# as a scripting language in game development	16
3. Methodology	19
3.1 Research and planning & requirements	20
3.2 Development and game testing	21
3.3 Evaluation and version management	22
4. Design of core gameplay systems	24
4.1 Core elements	24
4.2 Player locomotion and camera	26
4.3 UI and persistent game state	28
4.4 Colliders, animations and damage	29
4.5 Network synchronization	34
5. Results and Evaluation	36
5.1 Testing the system	37
5.2 Technical and design challenges	38
6. Conclusion	39
Literature	40
Keywords	41

Figure list

List Of Images

1. [Figure 1. Elden Ring](#)
2. [Figure 2. - Unity game object hierarchy](#)
3. [Figure 3. C# Logo](#)
4. [Figure 4. - Diagram](#)
5. [Figure 5. Player prefab](#)
6. [Figure 6. Memory usage analysis](#)
7. [Figure 7. Git logo](#)
8. [Figure 8. - Project start screen](#)
9. [Figure 9. - The starting construct](#)
10. [Figure 10. Blend tree](#)
11. [Figure 11. - Load game screen](#)
12. [Figure 12. YOU DIED event](#)
13. [Figure 13. - Broadsword model](#)
14. [Figure 14. - Body colliders](#)
15. [Figure 15. - Multiplayer network testing](#)
16. [Figure 16. Profiler CPU usage analysis](#)

1. Introduction

The goal of this research is to design and implement a playable prototype of a Soulslike action RPG using the Unity engine, with a focus on exploring how Unity can support the development of complex gameplay systems typical of the genre. The inspiration behind the project stems from a desire to understand what it takes to build a gameplay experience reminiscent of titles like Elden Ring or Dark Souls, whose design philosophy of "meaningful adversity" established the genre's identity, within the constraints of a smaller, personal development scope¹. Emphasis is put on challenging combat, environmental storytelling, and risk-reward progression as core pillars of the genre². Soulslike games intentionally incorporate hidden mechanics to encourage player exploration and foster discussion within the gaming community. This aligns with Miyazaki's approach to Dark Souls, where fragmented narratives and interconnected level design create a "rich, organic world" through collaborative artistry³. The genre's combat systems prioritize skill mastery over statistical advantage. FromSoftware's evolution, seen in Bloodborne's shift to aggressive playstyles, demonstrates how mechanical precision and thematic cohesion define the genre's appeal. From a technical perspective, Unity offers a robust platform for prototyping such systems due to its component-based architecture, support for custom scripting, and flexible animation and physics systems. Unity's prefab system and event-driven model are particularly well-suited for implementing modular gameplay mechanics, such as stamina-based combat, animation-priority actions, and dynamic UI feedback⁴.

This research draws upon both contemporary design analysis and practical development strategies. Bycer's Game Design Deep Dive: Soulslike provides a comprehensive examination of the genre's evolution, highlighting the significance of deliberate difficulty, environmental storytelling, and player-driven gameplay⁵. By developing a prototype that incorporates these elements, this study aims to

¹ Barnard Translations – 2018 Barnard Translations: Dark Souls II Design Works

² Bycer Joshua – 2024 Game Design Deep Dive: Soulslike – Ch. 1, P. 1

³ Barnard Translations – 2018 Barnard Translations: Dark Souls II Design Works

⁴ Tykoski Scott – 2021 Mastering Game Design with Unity – P. 4

⁵ Bycer Joshua – 2024 Game Design Deep Dive: Soulslike – Ch. 6, P. 62

contribute to the academic and practical understanding of Soulslike game development. It will evaluate Unity's strengths and limitations in supporting genre-specific features, such as body-accurate hitboxes, poise-based combat, and death mechanics, while also reflecting on the iterative process required to balance challenge, accessibility, and player engagement.

1.1 Objectives and scope

The main objective of the project is to design and implement a combat system prototype with the characteristics of Soulslike games and how Unity and its tools can help achieve that. A big part of the project is also to reflect on the solo developer process and identifying all the practices that helped maintain code quality and productivity. The expectation is to learn and document the practical process of game system design, tackling challenges in combat responsiveness and player experience. The prototype serves as both a technical case study and a learning project. The focus is on core gameplay mechanics and how they can be efficiently realized using Unity's features. The project evaluates how Unity empowers solo developers to create complex, genre-specific mechanics while also identifying any limitations or workarounds needed in building such systems independently. This evaluation encompasses not only technical feasibility but also considerations of scalability and adaptability. By identifying and analyzing the limitations and workarounds encountered during development, the project seeks to contribute practical insights for independent developers and to inform best practices for the use of Unity in similar contexts. It seeks to advance understanding of how solo developers can leverage modern game engines while also contributing to the broader discourse on independent game development and the role of technology in shaping player and game developer experience.

1.2 Sources of Data and Methods of Collection

The research and development process of this thesis is done with a diverse set of data, asset sources and methodological approaches. The foundation is constructed from a review of key principles in the field of game design, gathered by studying authoritative books and instructional videos to understand core concepts and best practices. A significant source of references for this project consisted of direct gameplay analysis of Soulslike games such as Dark Souls and Elden Ring, which provided inspiration and reference for key gameplay mechanics and design elements. These games serve as both inspiration and benchmarks, offering concrete examples of how abstract design principles are realized in practice. Through direct gameplay analysis, observation of player communities, and examination of developer interviews and discussions, the thesis identifies key features and recurring challenges associated with the Soulslike genre. Game assets were sourced from the Unity Asset Store to facilitate rapid prototyping and development, enabling a focus on gameplay systems over original asset creation. The methods of data and asset collection included hands-on experimentation within the Unity engine, where game systems were implemented, tested, and iteratively refined. Additionally, profiling tools such as the Unity Profiler were used to collect quantitative data on performance metrics like memory usage and frame rates. This data informed optimization strategies and ensured that the prototype met acceptable performance standards. Informal playtesting was also conducted, providing qualitative feedback on aspects such as gameplay feel, difficulty balancing, and overall user experience. Together, these approaches combined theoretical knowledge, practical implementation, and empirical testing to guide development decisions.

1.3 Content and Structure of the Thesis

The structure of this thesis is designed to guide the reader through the entire process of conceptualizing, developing, and evaluating a Soulslike action RPG prototype, while situating each stage within a broader theoretical and practical context. The thesis is divided into several main chapters, each addressing a distinct aspect of the project. The introduction outlines the subject and aims of the thesis, describes the sources and methods employed, and provides an overview of the thesis structure. Covering the foundational principles of game design, with a particular emphasis on the Action RPG and Soulslike genres, it reviews relevant literature, discusses the evolution of genre conventions, and identifies the core mechanics and design philosophies that distinguish Soulslike games from other action RPGs. The discussion highlights essential elements such as stamina-based combat, punishing difficulty, fluidity of combat and movement. Following this, the thesis examines the selection and capabilities of the Unity engine and supporting tools, explaining why Unity was chosen as the development platform. Special attention is given to features such as the component-based design model, animation systems, and the prefab system, as well as third-party tools that accelerated development. The next chapter covers the structured and systematic methodology applied to the project, detailing the iterative process of designing, implementing, testing, and refining gameplay features. The core of the thesis is the practical implementation of gameplay systems. Topics include player movement and locomotion, combat mechanics such as targeting and attack timing, user interface design to convey critical information, networked multiplayer synchronization for co-op gameplay, and game state persistence for saving progress. This chapter is supported by diagrams, code examples, and screenshots that illustrate design decisions and technical challenges. The thesis concludes with an evaluation of the prototype's performance and playability, discussing insights gained, limitations encountered, and potential directions for future work, thus bridging the gap between academic research and practical game development.

2. Background

Games are structured interactive experiences where players engage with defined objectives, challenges, and mechanics to achieve goals. Central to successful game design is the establishment of clear objectives that motivate players and provide a sense of purpose throughout gameplay. Well defined goals guide player actions and maintain engagement by offering meaningful progression and rewards. For example, in strategy games, the objective to build a thriving civilization directs player decisions and fosters investment in the game world. Core game mechanics the building blocks of gameplay and define the player's experience.

These mechanics create gameplay loops, repetitive sequences of actions that players perform to progress, such as exploration, combat, or resource management. Effective game design balances innovation in mechanics with consistent feedback and pacing to sustain player interest.⁶ Player agency (the degree of control and influence over the game world) is enhanced by mechanics that allow meaningful choices, increasing immersion and personal investment. Hybrid games that combine multiple gameplay styles appeal to diverse audiences by offering varied experiences, while adaptability in objectives and challenges supports different player preferences and playstyles. For example, open-world RPGs permit players to approach goals through combat, stealth, or diplomacy, enhancing replayability and player ownership. Game design is a deliberate process of crafting goals and mechanics to create engaging and rewarding player experiences. These principles underpin all genres and provide a foundation for understanding the specific design challenges and solutions in Action RPGs and Soulslike games.

⁶ Swink, Steve. *Game Feel: A Game Designer's Guide to Virtual Sensation*. CRC Press, 2009 – P. 40-60

2.1 Game development principles for Action RPGs

Action role-playing games (Action RPGs) combine real-time combat mechanics with character progression, exploration, and narrative elements. Developing such games requires balancing gameplay systems with responsiveness and immersion. The core mechanics are the data and algorithms that define the rules of play. They are the heart of the game, determining what is possible and how play proceed⁷. The development process focused on making more scalable, testable, and maintainable code especially for prototyping or expanding a game over time. Players should feel a sense of growth through leveling, acquiring new gear, and unlocking abilities. Well-designed progression systems balance power scaling with challenge, and offer meaningful customization that supports different playstyles. The game world must support exploration and discovery. In Soulslike Action RPGs, this often includes non-linear level design along with environmental storytelling, and visual landmarks. Environmental design plays a key role in both narrative delivery and gameplay pacing. Enemies and bosses must exhibit intelligent and varied behavior, using state machines, pathfinding, and animation-based attacks. The AI must be designed to challenge the player while behaving predictably enough to allow strategy and mastery. Clear feedback, both visual and audio, is essential. Players should know when they've landed a hit, taken damage, parried an attack, or triggered an ability. UI elements like health bars, stamina meters, and inventory screens must be intuitive and non-intrusive. To support real-time combat and dynamic events, Action RPGs rely heavily on scripting frameworks (like C# in Unity), often incorporating event-driven systems for animation states, damage calculation, and world changes (opening shortcuts or triggering boss fights).

⁷ Ernest Adams, Andrew Rollings – 2007 Fundamentals of Game Design – Ch. 7, P. 307

2.2 Characteristics of the “Souls-like” genre

The “Soulslike” genre is a subgenre of action RPGs that emerged primarily from the influence of FromSoftware’s *Demon’s Souls* (2009), *Dark Souls* (2011) and *Elden Ring*(2022)(Fig. 1). For over a decade, From Software defined a new genre that has led to studios chasing after them hit after hit⁸. These games introduced a distinct design philosophy and gameplay identity that have since been adopted and adapted by many developers. The genre is defined by a number of key characteristics that shape the player's experience and significantly influence gameplay systems. Soulslike games are renowned for their deliberate, methodical combat mechanics. Players must learn enemy patterns and react with precision, often choosing between light and heavy attacks, blocking, dodging, or parrying. Player actions such as attacking, blocking, and dodging consume stamina, which must be managed carefully. Overexertion leaves the player vulnerable. Health recovery is often limited, adding risk and tension.

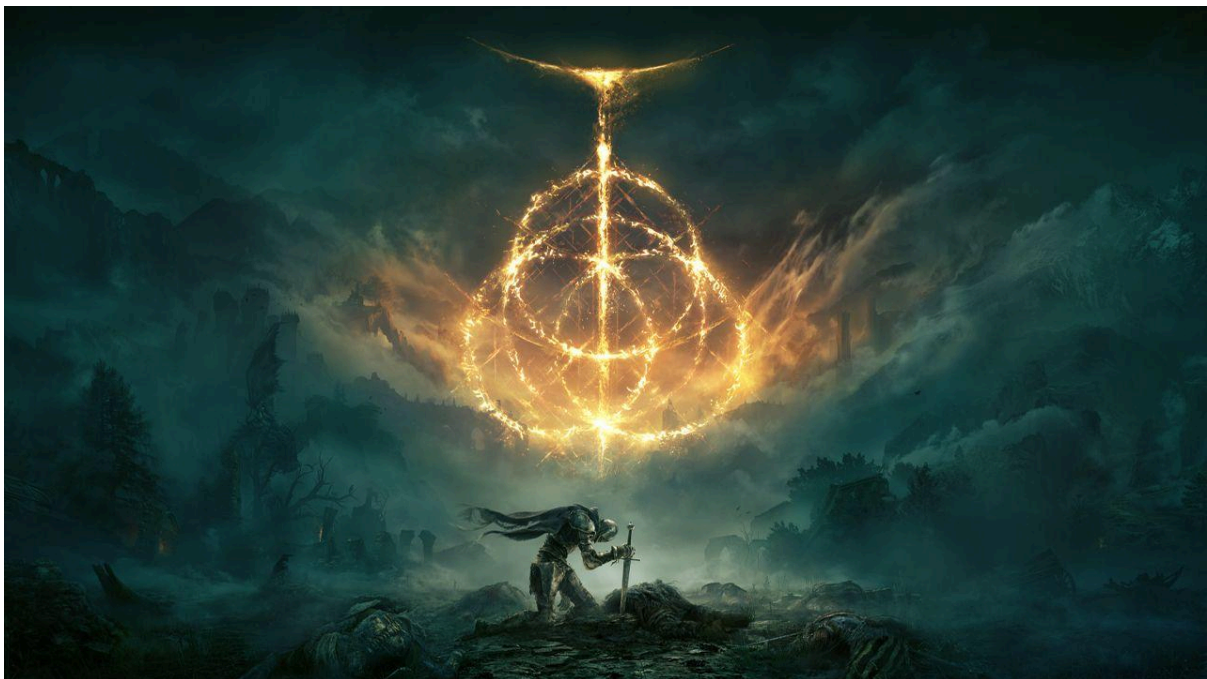


Figure 1. Elden Ring (source:

<https://www.deviantart.com/favorisxp/art/Elden-Ring-My-version-of-live-wallpaper-for-PC-916485601>)

⁸ Bycer Joshua – 2024 Game Design Deep Dive: Soulslike – Ch. 1, P. 1

As one of the core appeals of the genre is its difficulty, death is expected and often frequent, but it serves as a learning tool. Players typically drop their in-game currency or experience points upon death, which can be retrieved if they return to the location without dying again, creating high stakes for survival. This genre of games often features a dense, interconnected world with shortcut-based exploration, minimal hand-holding, and an emphasis on environmental storytelling. Navigation is part of the challenge and immersion. Narratives are often cryptic or implicit, delivered through item descriptions, environmental cues, and vague dialogue. Players piece together the lore through exploration and interpretation rather than exposition. Each area typically culminates in a difficult boss encounter, designed to test the player's understanding of the mechanics learned up to that point. Bosses are memorable, mechanically distinct, and often symbolic within the game's lore.

Features included are often character leveling, weapon upgrades, and class or build systems, allowing for various playstyles (e.g., melee-focused, ranged, magic). However, progression is often balanced to ensure skill remains central to success. The user interface is usually minimalistic, enhancing immersion. The game's mechanics are often communicated through animations, sounds, and visual cues rather than UI prompts or tutorials.

2.4 Overview of game engines

Game engines are comprehensive software frameworks designed to facilitate the development and creation of interactive digital games. They provide developers with a wide variety of tools and reusable components that handle core functionalities such as rendering graphics, managing physics simulations, processing input, handling audio, and orchestrating animations. By giving access to these complex systems, game engines enable developers to focus more on game design and content creation rather than low-level programming. Modern game engines vary in complexity and specialization, ranging from engines tailored for 2D mobile games to those capable of producing expansive 3D open-world experiences. Popular engines such as Unreal Engine, Godot, CryEngine, and Unity have become industry

standards due to their versatility, extensive feature sets, and active developer communities. These engines support multiple platforms, including PC, consoles, and mobile devices, thereby broadening the potential audience for games developed using them. The choice of a game engine is influenced by several factors, including the project's scope, target platform, team expertise, and desired features. For independent developers and small teams, accessibility, ease of use, and strong community support often weigh heavily in this decision. Engines that allow rapid prototyping and iteration are particularly valuable when developing complex gameplay systems or experimenting with novel mechanics. Among the available choices, Godot and Unity emerged as the two most practical and relevant candidates, both offering modern toolsets, active communities, and support for 3D game development. To better understand which engine aligns more closely with the needs of a Soulslike Action RPG prototype, a comparative analysis was made. The following table summarizes the most relevant similarities and differences between Godot and Unity in the context of this thesis:

Aspect	Godot	Unity
Ease of Use and Learning Curve	Simple, clean UI. Smaller ecosystem, so fewer Soulslike tutorials.	Steeper learning curve, a lot of marketplace assets. More built-in tools
Performance (3D)	Behind Unity in large-scale 3D worlds, occasional bottlenecks	Mature 3D engine with highly optimized pipelines.
Animation & State Machines	Animation tools are powerful. State machines require more manual setup.	Animator Controller and Timeline offer advanced state machines
Physics & Hitboxes	Node system makes it easy to layer colliders (hurtbox, hitbox). Physics engine is simpler but transparent.	PhysX-based engine, more tools for advanced collision, better third-party tools for hitbox visualization.

Networking / Co-op	Built-in networking. requires more manual coding.	More mature netcode, third party tools can make it easier to implement
Asset Pipeline & Marketplace	Smaller marketplace. Importing custom assets (animations, models) is straightforward.	Unity Asset Store has many Soulslike templates, stamina systems, combat kits, boss AI, etc.

Table 1. Comparison of Godot and Unity

Unity was chosen as the best engine for the purpose. Unity is a robust and versatile game development engine that is widely adopted for the creation of both 2D and 3D interactive experiences across a variety of platforms. Its accessibility, extensibility, and comprehensive toolset have made it a preferred choice for independent developers. It has the capacity to support the complex, interdependent systems characteristic of the genre. Unity’s architectural core is based on the GameObject–Component model, wherein every element within a Unity scene is instantiated as a GameObject(Fig. 2), and functional behavior is modularly added through components such as Rigidbody, Collider, Animator, and custom scripts. This modular approach is particularly well-suited to the iterative and experimental nature of game prototyping, enabling developers to rapidly assemble and modify gameplay systems⁹. The Animator system in Unity further enhances this flexibility, allowing for the implementation of sophisticated character animations through blend trees, state machines, and transitional logic. Such features are essential for Soulslike games, where precise timing of combat animations and input buffering are critical to gameplay fidelity and player immersion.

⁹ Unity Technologies – 2025 Unity Documentation

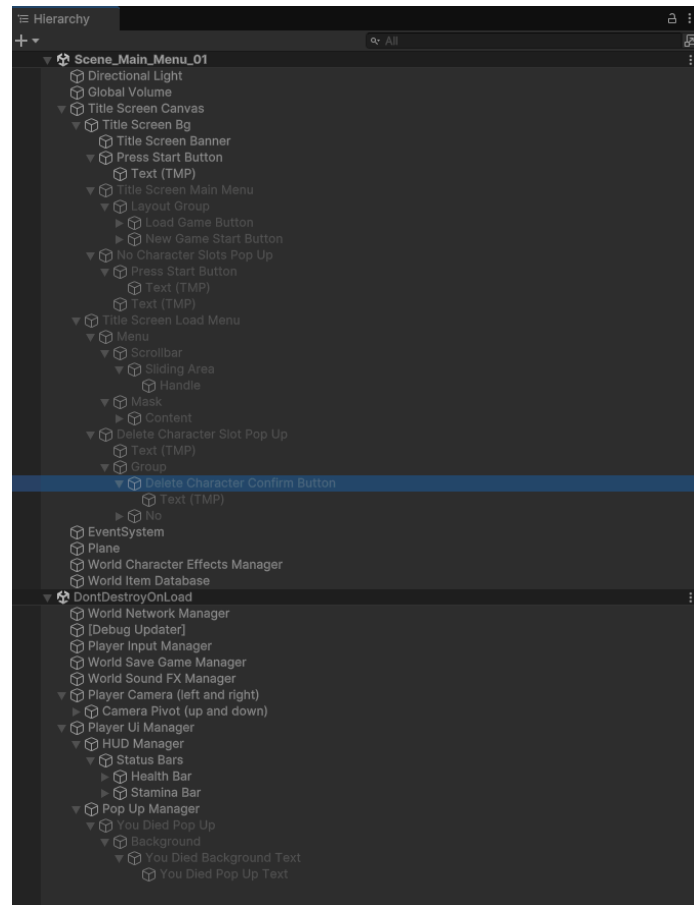


Figure 2. - Unity game object hierarchy (source: author)

Unity's use of C# as its primary scripting language affords developers the benefits of object-oriented programming and event handling. These capabilities enable the construction of complex systems. The Unity prefab system offers a means to create reusable object templates, which streamlines the management of enemies, weapons, and user interface elements throughout the development process. Prefabs provide consistency across game objects as well as level design and iterative testing, which is particularly advantageous during the prototyping phase. The Editor provides a big list of integrated tools, including the Inspector and Console windows, Profiler, and Play Mode, which collectively enable rapid iteration, real-time debugging, and performance optimization. These features are essential when developing a prototype with multiple interdependent systems, as they allow for immediate feedback and continuous refinement. Unity supports a broad spectrum of asset formats and maintains a robust Asset Store, wherein developers can access a wide array of models, textures, sounds, and development tools. This ecosystem

significantly accelerates the early stages of prototyping and production, as evidenced by the utilization of numerous third-party assets during the initial phases of this project. Accessibility, real-time feedback mechanisms, and comprehensive documentation render it an ideal platform for the development and testing of complex gameplay systems, such as those found in Soulslike RPGs. Its flexible architecture supports rapid iteration and enables deeper experimentation with gameplay mechanics, making it well-suited for both academic research and commercial game development.

2.5 C# as a scripting language in game development

C# is the principal programming language utilized within the Unity game engine. C# is recognized as a modern, object-oriented language that merges the robustness and efficiency of C++ with the safety and accessibility of languages such as Java. Within Unity, C# functions as the foundational scripting language, underpinning nearly all aspects of game logic and interactive behavior and C# scripts allow developers to create and control game objects, implement mechanics, and manage interactions between systems¹⁰. The selection and implementation of C# in this project were driven by its alignment with the requirements of a Soulslike Action RPG prototype, where reliability and maintainability are paramount. One of the primary advantages of C# in Unity development is its object-oriented programming (OOP) paradigm, which facilitates the organization of game elements into modular, reusable classes. This approach supports the creation of hierarchical structures, such as a base Enemy class with specialized subclasses for unique enemy types, promoting code reuse and reducing redundancy. Inheritance and polymorphism enable shared functionality, such as movement or attack logic, to be defined once and extended as needed, which is particularly beneficial in the context of a Soulslike game, where diverse enemy behaviors and modular systems are essential for engaging gameplay. C#'s strong typing system further enhances the development process by requiring variables to be explicitly defined with specific data

¹⁰ Ferrone Harrison – 2021 Learning C# by Developing Games with Unity 2021 – P. 45

types. This feature improves code reliability by catching type mismatches and other errors at compile time rather than during runtime, minimizing the risk of unexpected behavior in critical systems such as combat, stamina management, and AI logic. Such robustness is especially valuable in complex, interdependent systems, where early error detection can significantly reduce debugging time and improve overall project stability. The language's support for event-driven programming aligns seamlessly with Unity's component-based architecture. C#'s delegates and events provide a clean mechanism for system communication, allowing components to interact without tight coupling. In addition to the object-oriented design, an essential architectural element underpinning all real-time games, including those developed in Unity, is the Game Loop. The Game Loop is the central mechanism that drives the continuous execution of gameplay logic by repeatedly processing input and updating the game state. Within Unity, this loop is abstracted but effectively implemented through the `Update()`, `FixedUpdate()`, and `LateUpdate()` methods. The Game Loop is an important factor of interactivity and responsiveness, which are key points in Soulslike games where precise timing and reactive enemy AI significantly influence gameplay quality and player immersion.

For instance, a `HealthComponent` can trigger an `OnDeath` event, which the user interface (UI) system or AI logic can respond to, enabling dynamic gameplay reactions such as displaying a game over screen or spawning loot. This decoupled design fosters modularity and makes the codebase easier to maintain and extend. In terms of performance, while C# may not offer the same level of control as languages like C++, it delivers sufficient efficiency for the vast majority of gameplay scenarios in Unity-based projects. Unity's integrated profiling tools, such as the Profiler, allow developers to monitor CPU and memory usage in real time, identify bottlenecks, and optimize critical systems such as AI routines or animation logic to ensure smooth and responsive gameplay. C# is also distinguished by its clear syntax and strong community support. The language's readability facilitates collaboration among team members and simplifies the process of maintaining and updating code over time. Additionally, the extensive global community and wealth of online documentation, tutorials, and forums provide invaluable resources for troubleshooting and learning,

which is particularly advantageous during the rapid prototyping phase of development.

Integration with Unity's systems is another significant benefit. Since Unity's API is written in C#, developers have direct access to core engine features including physics, animation, and editor tools through intuitive scripting interfaces. This integration enables the straightforward implementation of complex mechanics, such as stamina regeneration systems, custom AI state machines, and interactive UI elements, while also supporting the creation of editor extensions for streamlined development workflows. The use of C# within Unity's component-based architecture provides a powerful and flexible foundation for developing complex, interdependent gameplay systems characteristic of Soulslike Action RPGs. Unity's seamless integration of scripting, animation, physics, and editor tools enables efficient prototyping and iterative development. Focusing on C# (Fig. 3) as the main scripting language in Unity and its suitability for the needs of a Soulslike Action RPG, forms a clear basis for the practical work ahead. Knowing how C#'s object-oriented design, event-driven features, and integration with Unity's component system work helps explain the choices made during development. This understanding guides the creation of modular and maintainable game systems, as well as the focus on performance and responsiveness. The following chapters will show how these principles were applied in building and testing the prototype.



Figure 3. C# Logo (source: https://commons.wikimedia.org/wiki/File:Csharp_Logo.png)

3. Methodology

The development of the Soulslike action RPG prototype in Unity was guided by a structured and systematic methodology designed to address the complexity of the genre and the technical demands of the project. Given the scope and solo nature of the development, a hybrid approach combining agile principles with iterative development was adopted to ensure flexibility, continuous improvement, and manageable workload.

To clearly illustrate the workflow and decision-making process, a custom methodology diagram was created (Fig. 4). This diagram visually maps out the iterative cycles of planning, prototyping, implementation, and evaluation that structured the development process. The design of this diagram was inspired by the iterative development model presented in the paper "Goal Oriented Agile Unified Process (GOAUP): An Educational Case Study," which emphasizes goal-driven phases and adaptability in software projects.¹¹

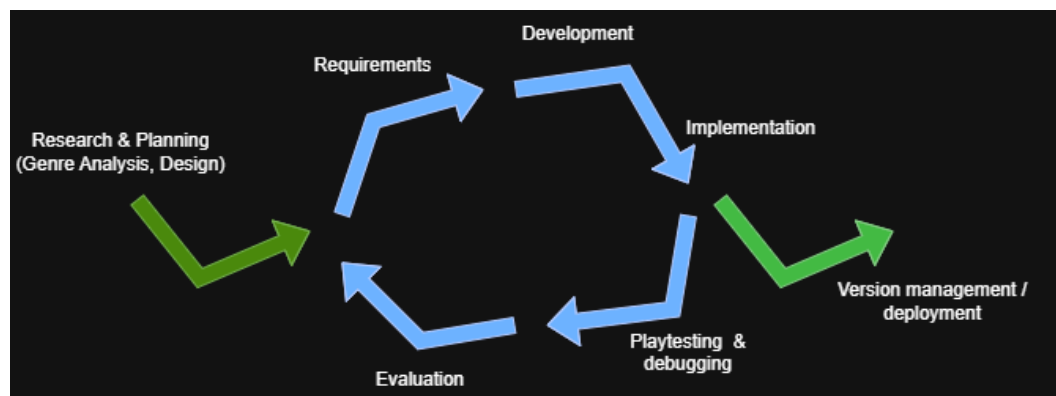


Figure 4. - Diagram (source: author)

¹¹Jun Lin, Chunyun Miao,Zhiqi Shen, Wei Sun – 2013 Goal oriented agile unified process (GOAUP): An educational case study 10.2991/icsecs-13.2013.9

3.1 Research and planning & requirements

The project commenced with an in-depth research phase aimed at deconstructing the core mechanics and design philosophies of Soulslike games. This phase involved analyzing key gameplay elements such as player locomotion, combat responsiveness, animation, network synchronization, and user interface design. This research was conducted through gameplay sessions, academic literature review, and developer interviews, providing a comprehensive understanding of the genre's expectations. This phase is also where the decision to build the game on Unity was made. Building upon the initial research and design planning, the project proceeded to define the essential requirements. These requirements were carefully formulated to capture the gameplay experience characteristic of the genre and the technical constraints of solo development within Unity. Functionally, the prototype needed to embody the deliberate and weighty feel of Soulslike combat, which required a stamina-based system for player attacks, defenses, and evasive maneuvers. This system had to ensure that player inputs translated into responsive yet meaningful actions, reflecting the genre's emphasis on timing and risk-reward mechanics. Complementing the combat, player locomotion was required to be fluid and immersive, supporting walking, running, dodging, and rolling animations that contributed to the overall gameplay feel. The user interface was designed to provide clear, real-time feedback on player status, particularly health and stamina, without cluttering the screen or detracting from immersion. Additionally, given the intent to explore multiplayer capabilities, the prototype needed to support network synchronization of player states and actions, ensuring consistent gameplay experiences across connected clients. On the technical side, performance considerations were an important factor. The prototype was expected to maintain a smooth frame rate on mid-range hardware, using efficient coding practices and optimization. Thorough notes and documentation of code and design decisions were emphasized to facilitate future development and debugging.

3.2 Development and game testing

Following the establishment of clear requirements, the development phase commenced with a focus on translating the conceptual design into a functional prototype within the Unity engine. Given the solo nature of the project, the development process was structured to balance progress with maintainability and iterative refinement. The development began with constructing the foundational systems, starting with player locomotion and combat mechanics. Emphasis was placed on leveraging Unity's component-based architecture to create modular, reusable scripts and prefabs (Fig. 5). This approach facilitated isolated development and testing of individual systems, reducing complexity and enabling focused troubleshooting.

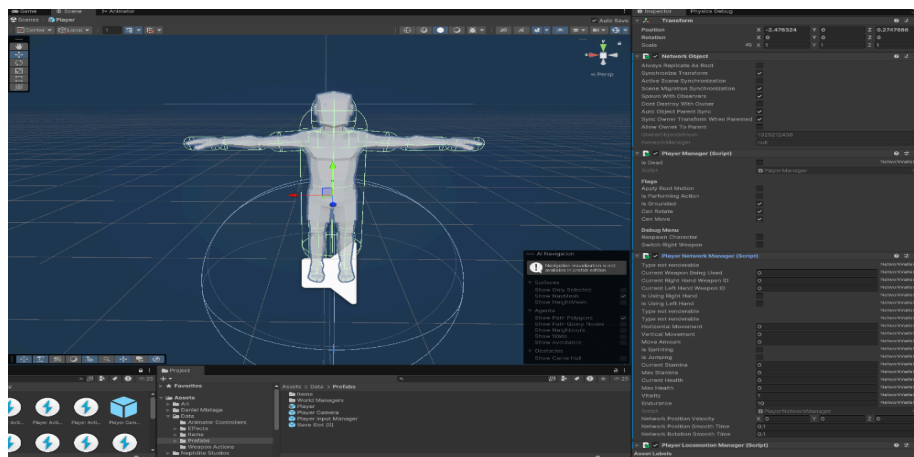


Figure 5. Player prefab (source: author)

The game testing and debugging phase was needed to validate the prototype's gameplay mechanics and ensure technical stability. Given the iterative nature of the development process, testing was conducted continuously alongside implementation to identify and resolve issues early. The primary focus of game testing was on evaluating combat responsiveness, stamina management, and overall player control fluidity. Feedback from these sessions highlighted areas where input latency, animation transitions, or stamina regeneration required fine-tuning to better replicate the deliberate and impactful feel characteristic of Soulslike games. Debugging efforts concentrated on resolving gameplay inconsistencies, fixing

glitches, and addressing synchronization errors in the multiplayer component. Unity’s built-in debugging tools, such as the Profiler and Console, were extensively used to monitor potential bugs.

3.3 Evaluation and version management

The evaluation phase served to critically assess the prototype’s effectiveness in meeting the defined requirements and achieving the intended gameplay experience. This phase involved both qualitative and quantitative analysis, drawing on data collected during playtesting as well as performance metrics.

Qualitative evaluation focused on player feedback regarding the combat system’s feel, the responsiveness of controls, and the overall immersion within the Soulslike framework. Observations were made on how well the prototype captured the deliberate pacing and challenge that define the genre. Particular attention was paid to the balance between difficulty and fairness, as well as the clarity of user interface elements. Quantitative evaluation utilized Unity’s profiling tools to measure frame rates and memory usage (Fig. 6). These metrics were essential in verifying that the prototype maintained stable performance within the target hardware specifications. Network latency and synchronization accuracy were analyzed to ensure a consistent multiplayer experience. The evaluation results highlighted strengths in modular system design and combat responsiveness.

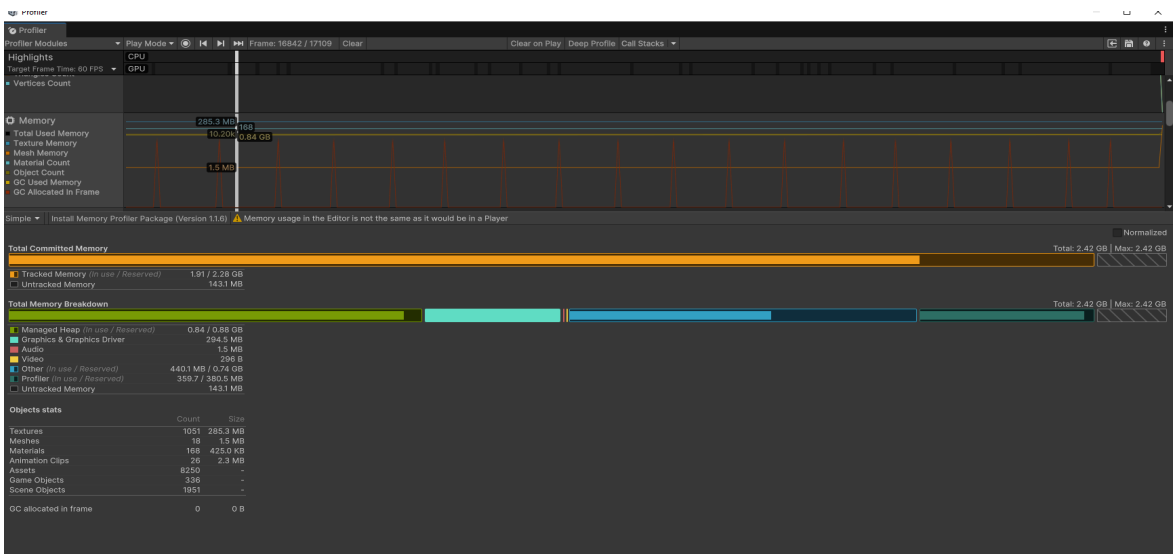


Figure 6. Memory usage analysis (source: author)

Effective version management was an important component of the project's development workflow, particularly given the iterative and experimental nature of the prototype's evolution. To maintain code integrity and continuous integration of new features, the Git version control system was used throughout the development process. Git (Fig. 7) enabled systematic tracking of changes, allowing the project to maintain a stable baseline while simultaneously supporting parallel development efforts such as feature additions, bug fixes, and performance optimizations.



Figure 7. Git logo (source:<https://commons.wikimedia.org/wiki/File:Git-logo.svg>)

Branching strategies were utilized to isolate experimental work, ensuring that unstable code did not compromise the main development line. This approach was especially valuable in a solo development context, where managing multiple development threads independently reduces the risk of errors and enhances productivity. When it comes to tools and technologies outside of Unity, Audacity was used for editing and managing sound effects like hits, enemy grunts, and UI feedback sounds. Temporary placeholder models were sourced from trusted asset libraries to simulate gameplay scenarios. Blender was used to make item and weapon models, as well as viewing and testing character animations. For the project management, Notion was used for task tracking, design documentation, and planning development sprints.

4. Design of core gameplay systems

The core gameplay systems are the most important part of the project, as they define the player's interaction with the world and form the foundation for the overall game experience. These systems shape the feel and flow of the gameplay which establishes the mechanical identity of the game. Their design demands careful attention to elements such as clarity and internal consistency, as well as responsiveness.

4.1 Core elements

Because a big part of the “Souls like” genre is being able to play with friends, a multiplayer-ready foundation was established through Unity's Netcode for GameObjects, allowing authoritative synchronization of player state and inputs across a networked environment. This proved to be a challenge and required attention throughout the whole development stage, but was essential for a good experience. The user interface (UI) plays a significant role in providing a polished and immersive experience. The title screen was implemented using Unity's UI Toolkit, with a canvas configured for modern screen resolutions (1920x1080). The title screen includes a background image with a game banner containing art made with the help of Paint3D, and a “Press Start” button, all positioned using anchors to ensure proper scaling across different displays. The “Press Start” button was designed to trigger the network initialization, mimicking the flow of commercial Souls-like games (Fig. 8).

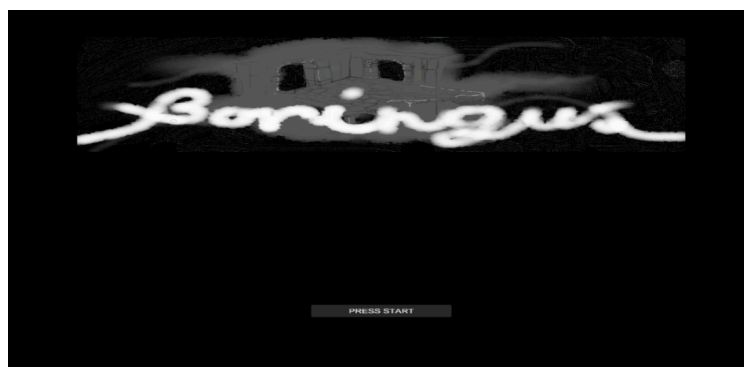


Figure 8. - Project start screen (source: author)

The initial phase of the project focused on establishing a solid foundation for networked gameplay. By leveraging Unity's built-in tools and adhering to best practices in project organization and scripting, the Souls-like game is well-positioned for future development and feature expansion, making sure everything is easy to access and defined very clearly.

To start the development and testing of core gameplay mechanics, a simple prototyping environment (Fig. 9) was constructed within Unity. This process began with the creation of a basic arena using primitive objects. A plane was instantiated to serve as the floor, and a custom texture was applied via a newly created material. Both textures and materials were stored in their respective folders within the "Art" directory, maintaining asset organization.

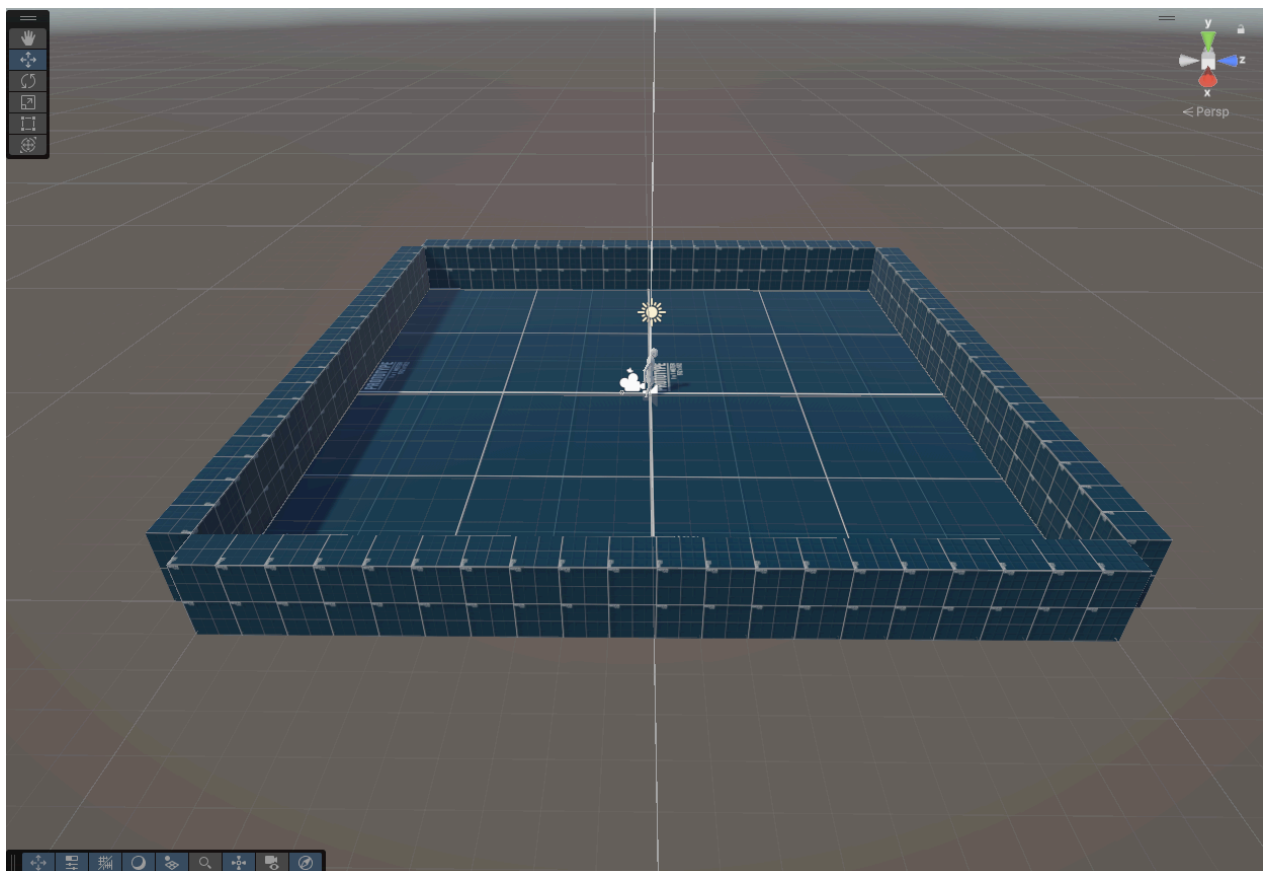


Figure 9. - The starting construct (source: author)

Walls were constructed using cubes, which were duplicated and arranged to form the boundaries of the arena. These cubes were parented under an empty "Walls" game object, itself nested within a "Construct" parent object. This structure kept the scene hierarchy organized and also enabled efficient manipulation and isolation of environment components. To prevent accidental selection and modification during scene editing, the "Construct" object was made unselectable in the editor, demonstrating attention to workflow efficiency. The next phase involves interfacing the input actions with gameplay scripts. The "Player Input Manager" script will be responsible for reading the values from the "Player Controls" asset and translating them into character movement within the game world.

4.2 Player locomotion and camera

Player movement was engineered around Unity's CharacterController to achieve responsive locomotion, with fine-tuned acceleration, deceleration, and friction profiles. Movement logic is encapsulated within a dedicated Locomotion Manager, which inherits from a base Character Locomotion Manager class. This architecture supports code reuse and future extension for both player-controlled and AI-controlled characters. This movement system integrated seamlessly with a third-person dynamic camera that featured adaptive offset, rotation smoothing, and a lock-on mechanism, providing clarity and control during exploration and combat. These systems were supported by an animation architecture utilizing blend trees and state machines. Animations for locomotion, combat, and recovery phases were precisely blended so that they match the pacing, with root motion selectively applied to preserve alignment with visual feedback.

The camera serves as the player's primary viewpoint and it significantly influences gameplay feel, spatial awareness, and immersion. To achieve smooth and natural camera motion, Unity's Vector3.SmoothDamp function was employed. This function interpolates the camera's position towards the player's position,

governed by a configurable smoothing parameter (cameraSmoothSpeed). The smoothing parameter is a serialized field, making real time adjustment with fine tuning within the Unity Inspector. The implementation ensures that camera movement is neither abrupt nor sluggish, so that visually the image the player sees is smooth and cohesive.

A blend tree was implemented (Fig. 10) within the Animator Controller to interpolate between multiple locomotion animations based on player input. The blend tree's parameters are dynamically updated via script, based on real-time player input. The "Horizontal" and "Vertical" values are mapped directly from the player's movement vector, ensuring that the animation state accurately reflects the intended direction and speed. The implementation of animation blend trees represents an important step in achieving fluid, multidirectional locomotion.

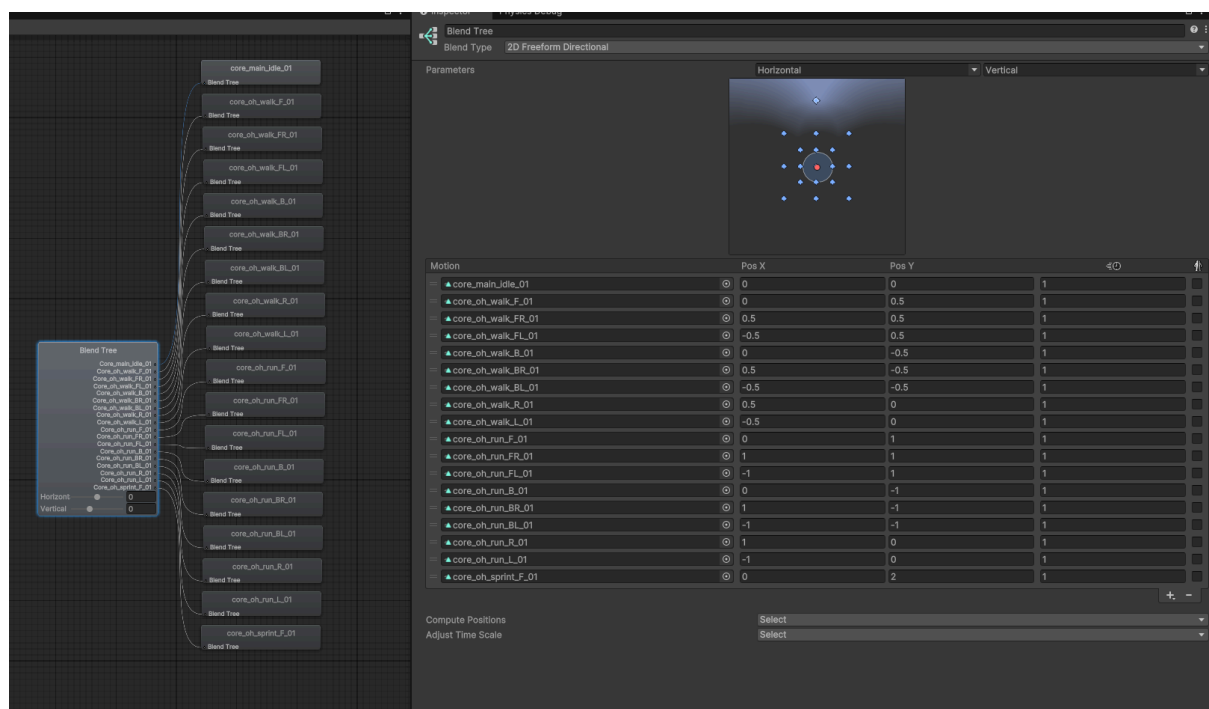


Figure 10. Blend tree (source: author)

An iconic feature of the Souls-like genre is its nuanced evasion mechanics, notably the roll and backstep actions that grant players precise control over their movement and survivability. Rolling and backstepping were implemented with directional input sensitivity and frame based invincibility states, which are the methods the player uses to avoid being damaged while doing these actions. These systems were

designed to reward player timing and spatial awareness. Sprinting was layered into the movement system, consuming stamina and adjusting animation playback to preserve physicality and visual coherence. Stamina consumption was made configurable through ScriptableObjects. Network syncs have also been made for all the actions.

4.3 UI and persistent game state

The stamina and health system's visual representation is anchored in the game's UI, necessitating a robust and scalable canvas setup. A hierarchical structure is established within the canvas to manage different UI modules. An empty GameObject, "HUD Manager" is created to serve as the parent for all heads-up display elements, such as status bars and equipment screens. Within the HUD Manager, a "Status Bars" object is anchored to the top-left corner of the screen, housing the health and stamina bars. Dynamic resizing of the stamina bar based on the player's endurance stat is present with the addition of visual feedback on recent stamina consumption. Another UI feature present during gameplay is the item and weapon slot UI, displaying the current weapon or item while being able to sort through them. These features that are inspired by the standards of core gameplay in games like Dark Souls and Elden Ring, reinforce the sense of progression and provide immediate feedback on player actions. To support multiplayer functionality, UI updates are gated by ownership checks, ensuring that each player only updates their own UI elements. This prevents cross-player interference and maintains a consistent user experience in networked sessions.

Persistent game state is a fundamental requirement for modern video games, particularly in the Souls-like genre, where player progress, character statistics, and world state must be reliably stored and retrieved across play sessions. The implementation of a JSON-based save system in Unity provides a reliable and extensible foundation for persistent game state management in a Souls-like video game. Through careful organization and file handling, the system ensures that player progress and character data are preserved across sessions (Fig. 11). The

system provides a seamless and intuitive experience for players managing multiple characters with a character selection menu.

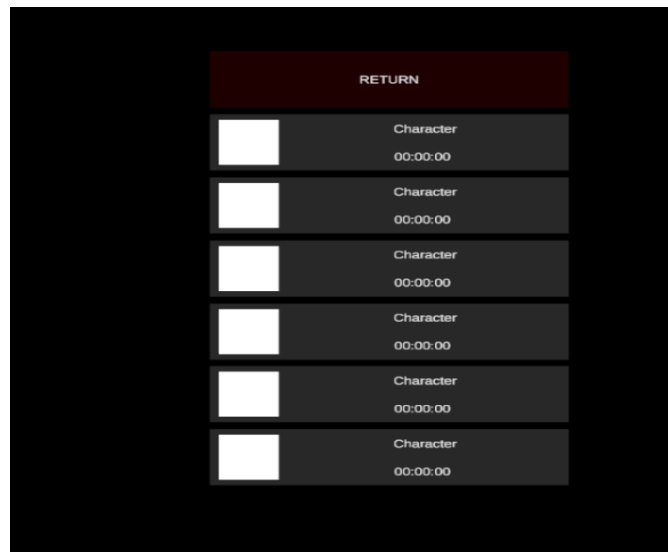


Figure 11. - Load game screen (source: author)

Vertical traversal and falling mechanics were introduced with physics-aware logic and grounded checks (which were also useful for many other checks), while visual and functional consequences (such as fall damage) added realism and risk to navigation. To achieve a fluid and realistic jumping sequence, the jump animation is divided into four segments: start, lift, idle, and end. This segmentation allows for precise control over the character's movement and visual feedback at each phase of the jump. The start animation initiates the jump, the lift applies vertical velocity, the idle loops while airborne, and the end triggers upon landing.

4.4 Colliders, animations and damage

Damage systems are central to the Souls-like genre, directly influencing combat dynamics, player survival, and overall game balance. The foundation of the damage system is the `TakeDamageEffect` script, which inherits from a base `InstantCharacterEffect` class. This inheritance structure allows for consistent processing of instant effects and supports future extension for other effect types. The script overrides the `ProcessEffect` method, encapsulating all logic related to

damage application, including animation, sound, and stat modification.

A simplified pseudo code snippet displaying the form of the TakeDamageEffect class and one of its methods (calculateDamage) -

```
CLASS TakeDamageEffect : InstantCharacterEffect

    VARIABLES:

        characterCausingDamage

        damageValues (physical, magic, fire, etc.)

        finalDamageDealt

        animationSettings

        soundSettings

        hitDirection

        poiseValues

    METHOD ProcessEffect(character)

        CALL base.ProcessEffect(character)

        IF character is dead THEN

            RETURN

        CALL CalculateDamage(character)

    END METHOD

    METHOD CalculateDamage(character)

        IF character is not controlled locally THEN

            RETURN

        IF final damage <= 0 THEN

            final damage = 1

        END METHOD

    END CLASS
```

Damage is applied via colliders attached to weapons, spells, or environmental hazards. These colliders are configured as triggers, allowing characters to pass through them while still registering hits. The DamageCollider script serves as the base class for all damage sources, with derived classes providing specialized behavior. The health bar UI is updated in real time to reflect changes in the player's health. Debugging tools, such as logging final damage values and using "gizmos" to draw the outlines of collider boxes, are employed during development to verify system behavior and diagnose issues, these are a must with any delicate system like the damage and collider logic, since knowing exactly what happens when is essential to bug fixing. Player death is a defining moment in Souls-like games, serving both as a gameplay reset and as a dramatic punctuation to the player's journey.



Figure 12. YOU DIED event (source: author)

To reinforce the genre's signature tension and feedback, this section details the integration of a death animation and a dedicated "YOU DIED" screen UI, triggered when the player's health reaches zero(Fig. 12). Upon entering the death state, a death animation is played. This is facilitated by the character's animator controller, which includes a dedicated "Death" state. The transition to this state is

controlled by the isDead parameter, ensuring that the animation plays immediately and reliably across the network. A new UI canvas is introduced specifically for the “YOU DIED” screen. After a brief delay or upon player input, the “YOU DIED” screen is dismissed, and player input is re-enabled. The game then proceeds to the respawn logic, which is relatively simple. Weapons are managed through an inventory system and equipped to the player via designated slots, dynamically updating the player’s stats and animations. This system is realised with a script that supports multiple “item” types such as consumables(One time use items that grant certain effects), talismans(items that give permanent effects as long as they are equipped) and weapons(Fig. 13) which have their own categories, such as “main hand” or “off hand” and weapon types like swords, spears or shields which all have their own functions. Weapon switching is triggered by player input, seamlessly swapping between equipped weapons by detaching the current weapon and attaching the new one, while ensuring the UI and animations reflect the change.

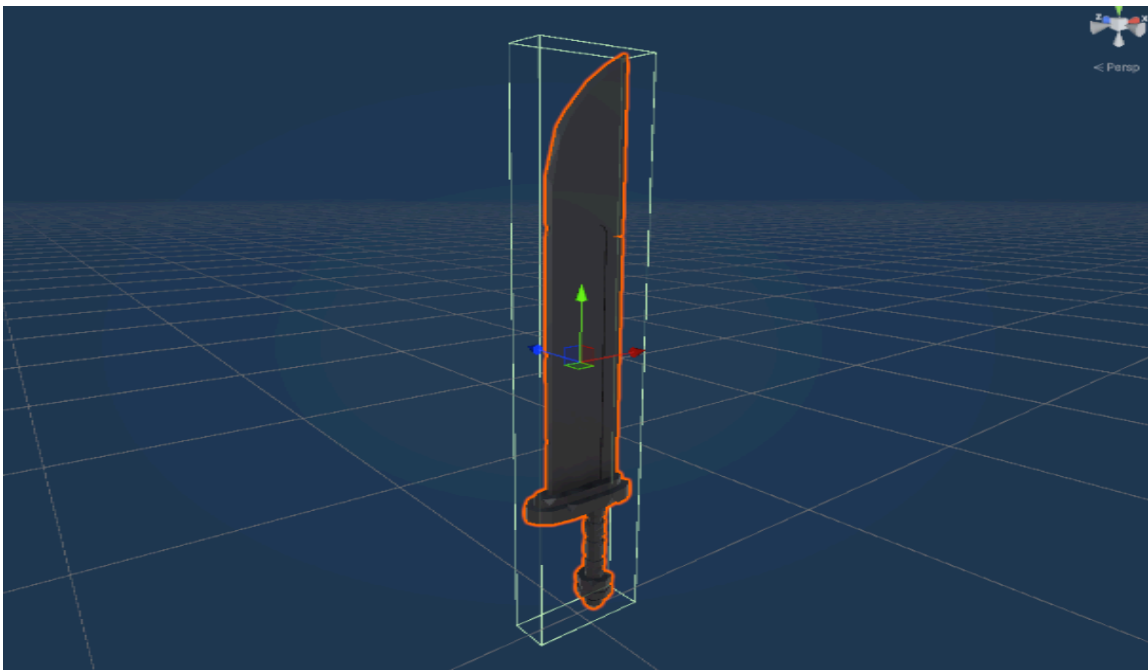


Figure 13. - Broadsword model (source: author)

Basic weapon actions consist of light attacks which are quick and efficient, while heavy attacks deliver more damage at the cost of speed. Parry allows players to deflect enemy attacks and counter with critical strikes. Combat is enhanced by body-accurate hitboxes and hurtboxes, enabling precise hit detection on different body parts and ensuring attacks only register during the correct animation frames. This system provides immersive skill-based combat, with modular support for future features and varied weapon movesets. Special properties, such as damage type, attack speed, range, and even elemental effects were assigned to weapons via scriptable objects or custom scripts. Body accurate hit detection is present through careful positioning of colliders in each part of the player “skeleton” (Fig. 14).

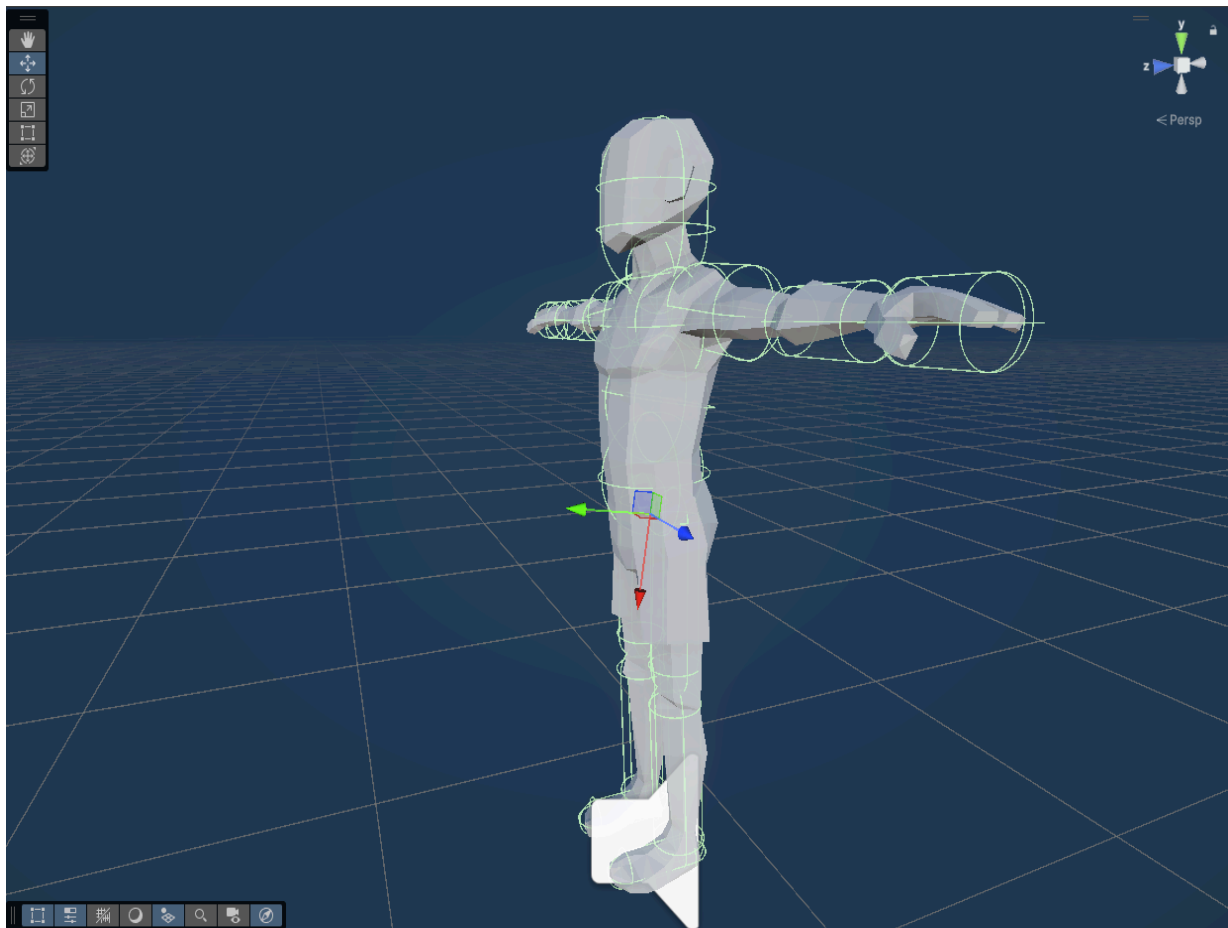


Figure 14. - Body colliders (source: author)

4.5 Network synchronization

Synchronized multiplayer functionality is essential to maintaining fairness and mechanical consistency in networked Souls like experiences. To achieve this, the prototype leverages Unity's Netcode for GameObjects to transmit player state, actions, and equipment across the network, ensuring each client has an up-to-date representation of other players. The `CharacterNetworkManager` class serves as the base network layer for all characters. It handles the real-time synchronization of core character data, including position, rotation, movement variables (horizontal, vertical, and move amount), and key gameplay flags such as sprinting and jumping states. To preserve smoothness despite network latency, position and rotation data are interpolated locally using smoothing parameters, preventing abrupt motion and enhancing the sense of responsive multiplayer play. The system also maintains vital gameplay resources such as current and maximum health and stamina within network variables.

The system also introduces a server-authoritative architecture for action animations. When a client triggers an action, like an attack animation, a `ServerRpc` is invoked to inform the server of the intended action. The server then relays this command to all other clients using a `ClientRpc`. This design prevents potential desynchronization or cheating by ensuring that all action animations originate from the server, while the initiating client skips re-execution locally to avoid duplicate animation playback. Root motion flags, which control whether an animation physically moves the character, are also transmitted, preserving the integrity of complex combat sequences across the network. The `PlayerNetworkManager` class extends `CharacterNetworkManager` by adding player specific synchronization logic, particularly focused on the equipment system. The player's character name is synchronized to all clients via a network variable, providing visual and social identification within the game world. Weapon and equipment state like the currently equipped right hand and left hand weapons and which hand is currently active are also maintained through network variables. When these variables change, callback methods instantiate the appropriate weapon objects from a centralized

WorldItemDataBase and assign them to the player's equipment manager. This dynamic replacement ensures that weapon changes made by the owning player are accurately reflected across all clients, supporting gameplay features like weapon switching and dual-wielding that are hallmarks of the genre. The system dynamically recalculates maximum health and stamina values when player stats such as vitality and endurance are increased. This process uses methods in the player's stats manager to ensure new stat levels produce consistent gameplay effects. The recalculated values are then updated in the player's UI through dedicated HUD manager methods, ensuring that changes made by leveling up or using stat-altering items are immediately visible and mechanically impactful.

Through this design, the network code maintains responsiveness and security. By isolating authoritative actions on the server while allowing cosmetic and non-critical updates to occur locally, the system simulates the fair yet fluid Souls-like multiplayer functionalities.

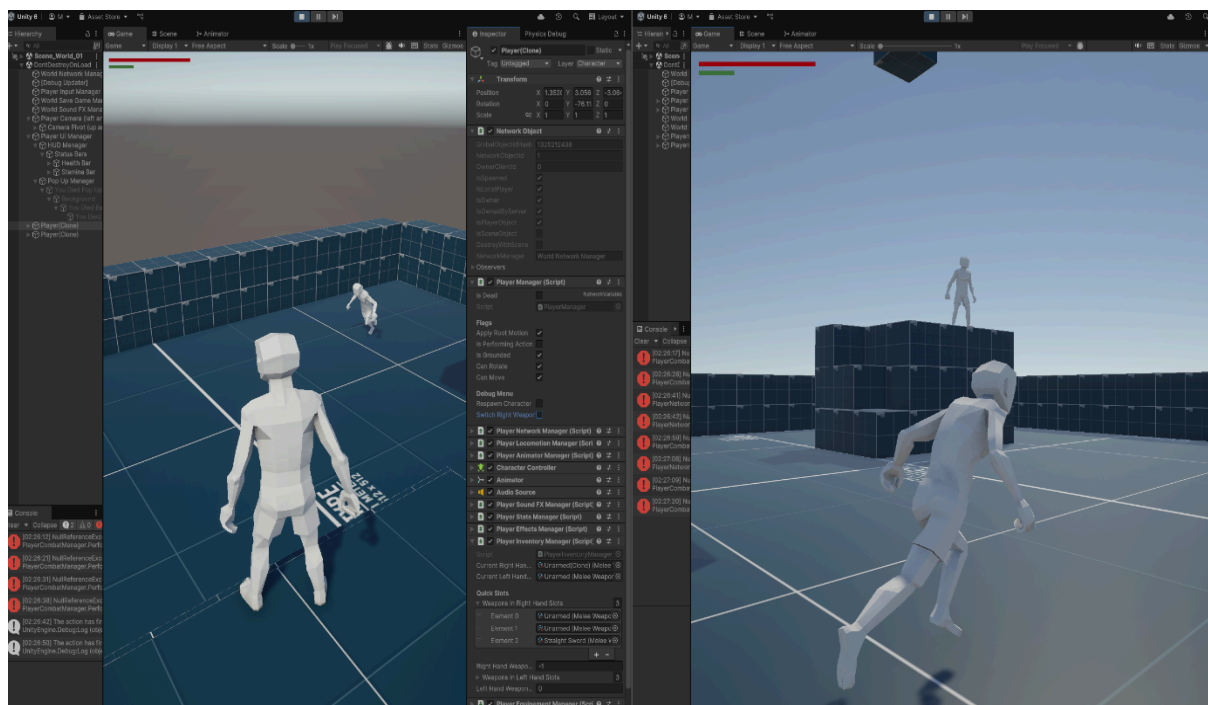


Figure 15. - Multiplayer network testing (source: author)

5. Results and Evaluation

The prototype underwent extensive manual testing to verify the functionality and stability of core gameplay systems. Each gameplay mechanic and stage of development was followed with batches of evaluations. The testing process involved multiple playthroughs to identify bugs, ensure consistent behavior, and validate player feedback mechanisms (Fig. 15). A helpful feature that was implemented during the development is the UI for game testing, which was a manual way to change values, states, resources or inventory. Being a developer tool for testing, This feature is only available in the unity editor and does not appear in gameplay. This is where a solo developer can spend a big chunk of their time, but valuably so since it is a learning experience.

Key results showed that the health and stamina systems accurately tracked resource changes and synchronized properly with the UI indicators. Combat hit detection and damage calculation responded reliably to player inputs, allowing for fluid and precise control. Enemy AI demonstrated expected behaviors such as patrolling, chasing, and attacking, creating dynamic combat scenarios. The boss encounter implemented phased attacks that scaled difficulty appropriately. While automated testing was limited due to time and resource constraints, iterative manual testing enabled rapid bug identification and resolution, contributing significantly to the legitimacy of the prototype. Responsiveness was measured through performance observations and subjective gameplay feel, focusing on the critical requirements of the Soulslike genre. The prototype consistently achieved stable frame rates exceeding 60 FPS on recommended hardware, ensuring smooth animations and fluid player movement. Input latency tests indicated minimal delay between player actions and in-game responses, which is essential for maintaining the precision expected in challenging combat scenarios. Of course, all of these features were synced between players in the netcode, and multiplayer proved to be functional.

5.1 Testing the system

In addition to iterative manual testing, the Unity Profiler was used to systematically monitor and evaluate the runtime performance of the prototype. The memory profiling snapshot reveals that the total committed memory reached approximately 2.42 GB, with around 1.91 GB actively tracked by Unity. A significant portion of this was allocated to the managed heap, which was close to its reserved limit of 0.88 GB. Textures represented the largest asset memory footprint, occupying around 285 MB, followed by materials and meshes. Additionally, the Graphics & Graphics Driver accounted for roughly 295 MB, reflecting the cost of real-time rendering operations. The profiler timeline further illustrates CPU utilization and frame distribution, where the RenderLoop on the render thread typically consumed about 5 ms per frame. Overall, the game maintained an average target of 60 FPS, with relatively stable frame timings and no major spikes that would suggest frequent garbage collection or CPU-bound bottlenecks (Fig. 16).

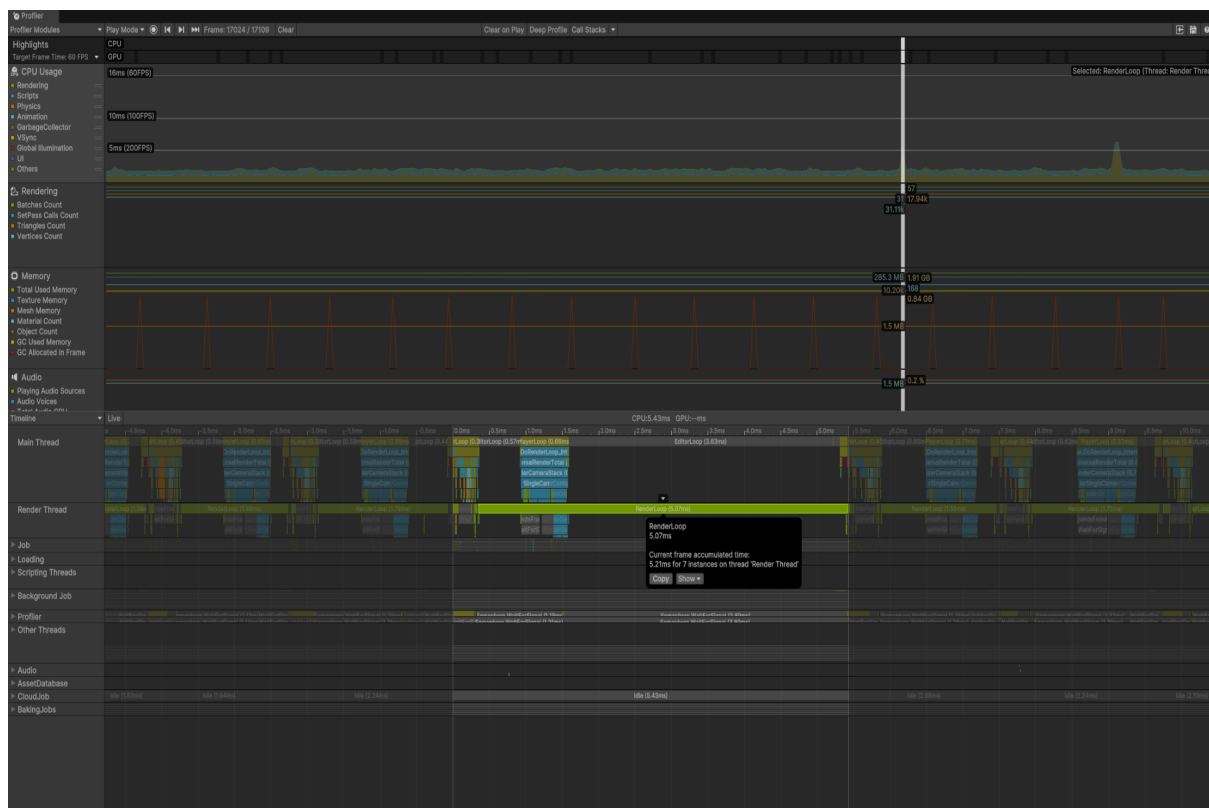


Figure 16. Profiler CPU usage analysis (source: author)

5.2 Technical and design challenges

Several technical and design challenges emerged throughout the development process. Hit detection accuracy and balancing collision detection to provide fair but challenging combat required many adjustments of hitboxes and timing windows, together with meshing animations and certain scripts that handle propulsion and movement. Designing enemy AI to feel intelligent yet predictable involved fine tuning state machines and behavior parameters to avoid frustrating or unrealistic enemy actions. Synchronizing complex animations, including attack combos, dodges, and damage reactions, while maintaining smooth transitions presented significant technical complexity. Another limitation was optimal asset creation or finding. While Unity does have a large number of assets meant for action RPGs, it won't always be exactly what the project needs, and creating such assets is a big time sink.

Debug logs proved to be very important in the stage of bug fixing and they saved many long attempts of fixes. Addressing these challenges needed frequent testing, code refactoring, and design iteration, ultimately improving the system's reliability and maintainability. Although achieving its core objectives, the prototype is subject to several limitations inherent to its scope, development constraints, and the iterative nature of the project. These limitations provide valuable insight for future development and highlight areas where the project could be expanded or refined. It was made to be very easily built upon and upgraded and so it is ready for the smooth implementation of many features such as interlinking levels, class systems, mutli-stage bosses and such. Further refinement of the user interface and feedback systems is essential. This includes polishing visual and auditory cues, adding dynamic UI elements, and ensuring that all feedback is clear and consistent. It also adds a layer of immersion, another big thing that can be reinforced. While the prototype includes a networked foundation, future work could expand multiplayer functionality to support cooperative or competitive play, leaderboards, and social features. This would align the project more closely with contemporary online gaming trends and could set itself apart from other Souls like multiplayer systems.

6. Conclusion

Undertaking the development of a Soulslike action RPG prototype in Unity has been both a technical challenge and a deeply educational experience. This project has illuminated not only the complexities of modern game design but also the creative and analytical mindset required to transform abstract genre conventions into tangible, interactive systems. Through the iterative process of building, testing, and refining core mechanics. Throughout the development process, this project has shown the value of adaptability, reflection, and iterative learning. Each obstacle encountered, whether technical, such as synchronizing networked gameplay, or design-related, such as balancing combat feedback, has provided practical lessons in problem-solving.

The limitations of the prototype, rather than lowering its worth, have highlighted the realities of solo development and underscored the importance of focusing on foundational experiences centered around the player. By prioritizing core mechanics and maintaining a clear, documented workflow, the project has demonstrated that even within resource constraints, it is possible to create a robust and engaging gameplay foundation. The experience has revealed both the strengths and limitations of Unity as a platform for genre specific development. The engine's flexibility and comprehensive toolset have proven valuable for rapid prototyping and iterative refinement, while its learning curve and certain technical constraints have necessitated creative workarounds and ongoing adaptation. This project stands as a testament to the potential of focused, iterative development and the empowering nature of accessible game engines like Unity. While the prototype may not yet rival the depth and polish of commercial Soulslike titles, it serves as a compelling proof of concept for what independent developers can achieve with dedication, thoughtful design, and a commitment to learning.

Literature

Bycer, Joshua. 2024. "Game Design Deep Dive: Soulslike." In *Game Design Deep Dive: Soulslike*, by Joshua Bycer, 1.

Ernest Adams, Andrew Rollings. 2007. "Fundamentals of Game Design." In *Fundamentals of Game Design*, by Andrew Rollings Ernest Adams, 307. California: Pearson Prentice Hall, 2007.

Ferrone, Harrison. 2021. "Learning C# by Developing Games with Unity 2021." In *Learning C# by Developing Games with Unity 2021*, by Harrison Ferrone, 45.

Technologies, Unity. 2025. Unity Documentation. Accessed May 23 2025. <https://docs.unity3d.com/Manual/GameObjects.html>.

translations, Barnard. 2018. *BARNARD TRANSLATIONS*.
<https://www.barnardtranslations.com/post/dark-souls-design-works>.

Tykoski, Scott. 2021. "Mastering Game Design with Unity 2021." In *Mastering Game Design with Unity 2021*, by Scott Tykoski, 4.

Swink, Steve. *Game Feel: A Game Designer's Guide to Virtual Sensation*. CRC Press, 2009. , 40-60, 120-140

.Lin, J., Miao, C., Shen, Z., & Sun, W. (2013). *Goal oriented agile unified process (GOAUP): An educational case study*. 10.2991/icsecs-13.2013.9

Keywords

1. Soulslike – Subgenre of action RPGs with challenging, methodical combat and player perseverance.
2. Action RPG – Role-playing games with real-time combat and character progression.
3. Unity Engine – Game development platform used for prototyping and building the prototype.
4. Game Prototyping – Creating early playable versions to test and refine ideas.
5. Iterative Development – Repeated cycles of building, testing, and improving systems.
6. Solo Development – Independent creation of the game by a single developer.
7. Game System Design – Crafting rules and mechanics that define gameplay.
8. Core Gameplay Mechanics – Fundamental systems like combat, stamina, and movement.
9. Combat Responsiveness – Immediate and fluid translation of player input into actions.
10. User Interface (UI) – Visual elements that communicate information to the player.
11. Networked Multiplayer – Online play with synchronized player actions.
12. Persistent Game State – Saving and loading player progress and world state.
13. Workflow Efficiency – Optimizing productivity and organization in development.
14. Narrative Cohesion – Integrating story elements with gameplay.
15. Player Experience – The emotional and cognitive engagement of the player.
16. Challenge and Difficulty – Design elements that create adversity and learning.
17. Community Collaboration – Players sharing knowledge and strategies.
18. Blend Trees – Animation technique for smooth transitions between actions.
19. ScriptableObjects – Unity data containers for modular configurations.
20. Hit Detection – Systems for registering combat interactions.
21. Event-Driven Programming – Logic that responds to triggered events.

22. Serialization – Saving and transferring game data efficiently.
23. Invincibility Frames – Temporary immunity during certain animations.
24. Player Locomotion – Movement systems for the player character.
25. Animation State Machine – Controls transitions between animation states.
26. Stamina System – Mechanic limiting actions through a stamina resource.
27. Health System – Tracks and displays player health.
28. Enemy AI – Artificial intelligence controlling enemy behaviors.
29. Level Design – Structuring environments and challenges.
30. Environmental Storytelling – Narrative delivered through the game world.
31. Risk-Reward Mechanics – Systems balancing player risk and reward.
32. Player Feedback – Visual, audio, or haptic responses to actions.
33. Multiplayer Synchronization – Keeping game state consistent across players.
34. Canvas System – Unity's system for organizing and rendering UI elements.
35. Network Initialization – The process of setting up network connections and player states.
36. Prototyping Environment – A simplified scene used to test core gameplay mechanics.
37. Game Object Hierarchy – The organizational structure of objects within a Unity scene.
38. Player Input Manager – Script that handles player inputs and translates them into actions.
39. CharacterController – Unity component that manages player movement and collision.
40. Locomotion System – The system controlling player movement mechanics.
41. Animator State Machine – Logic that controls animation states and transitions.
42. Root Motion – Animation-driven movement applied to characters for realism.
43. Third-Person Camera – Camera system following the player from behind.
44. Vector3.SmoothDamp – Unity function used to smoothly interpolate camera movement.
45. Lock-On Mechanism – Targeting system that focuses the camera and attacks on enemies.

46. Directional Rolling – Dodge mechanic allowing rolls in specific directions.
47. Frame-Based Invincibility – Temporary immunity frames during specific animation periods.
48. Sprinting Mechanic – Movement mode consuming stamina for faster traversal.
49. Health and Stamina Bars – UI elements showing player's health and stamina levels.
50. HUD Manager – Controller object managing heads-up display elements.
51. Dynamic UI Resizing – UI adapting size and layout based on screen resolution or stats.
52. Ownership Checks (Multiplayer UI) – Ensuring UI updates only affect the local player.
53. JSON Save System – Data format and system used to persist player progress.
54. Character Selection Menu – UI allowing players to choose saved characters.
55. Vertical Traversal – Mechanics for moving up and down in the environment.
56. Falling Mechanics – Logic governing falling and related consequences like damage.
57. Grounded Checks – Determining if the player is on the ground or airborne.
58. Fall Damage – Health penalty applied when falling from heights.
59. Netcode – The program code that handles interactions over a network
60. Damage System – Logic handling application and calculation of damage.
61. TakeDamageEffect Script – Script managing how damage affects characters.
62. InstantCharacterEffect Class – Base class for immediate effects like damage.
63. ProcessEffect Method – Function executing effect logic.
64. Damage Colliders – Trigger colliders that detect hits and apply damage.
65. Trigger Colliders – Colliders that detect overlaps without physical collision.
66. Health Bar UI Updates – Real-time visual updates reflecting health changes.
67. Debugging Tools – Methods and utilities used to identify and fix bugs.
68. Death Animation – Animation played when the player dies.
69. "YOU DIED" Screen – UI shown upon player death to indicate failure.

- 70. Animator Parameter Control – Variables controlling animation transitions.
- 71. Respawn Logic – Systems managing player revival after death.
- 72. Inventory System – Managing player equipment and items.
- 73. Weapon Slots – Designated attachment points for weapons on the player.
- 74. Weapon Switching – Mechanic allowing players to change equipped weapons.
- 75. Weapon Animations – Animations specific to weapon attacks and handling.
- 76. Light and Heavy Attacks – Different attack types varying in speed and damage.
- 77. Parry Mechanic – Defensive move to deflect attacks and counterattack.
- 78. Hitboxes and Hurtboxes – Colliders defining attack and damageable areas.
- 79. Body-Accurate Hit Detection – Precise hit detection based on body parts.
- 80. Weapon Properties via ScriptableObjects – Configurable weapon stats stored modularly.
- 81. Damage Types – Categories of damage like physical or elemental.
- 82. Attack Speed – Rate at which attacks are performed.
- 83. Attack Range – Distance an attack can reach.
- 84. Elemental Effects – Additional damage or status effects based on elements.
- 85. Greatsword Moveset – Specific attack patterns for a heavy weapon.
- 86. Dagger Moveset – Attack patterns for a fast, light weapon.
- 87. Spear Thrust Animation – Specific animation for spear attacks.
- 88. Body Colliders – Colliders attached to character body parts for hit detection.
- 89. Player Skeleton Setup – Hierarchical structure of bones for animation.
- 90. Modular Gameplay Architecture – Designing systems for easy reuse.
- 91. Event-Driven Programming – Programming model reacting to events or triggers.
- 92. Performance Optimization – Techniques to improve game efficiency.
- 93. Version Control (Git) – Managing changes to code and assets over time.
- 94. Agile Development Methodology – Iterative, flexible project management approach.
- 95. Playtesting – Testing gameplay with users to gather feedback.

- 96. Debugging Process – Systematic identification and fixing of bugs.
- 97. Combat Responsiveness – How quickly and smoothly player inputs affect gameplay.
- 98. Network Latency Handling – Managing delays in multiplayer communication.
- 99. Multiplayer State Management – Keeping player states consistent across network.
- 100. Animation Blending Techniques – Methods to transition animations smoothly.



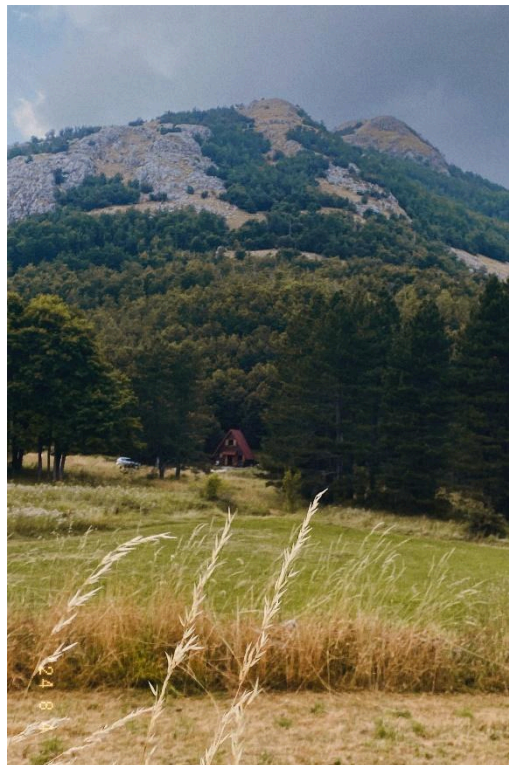
Relaxing



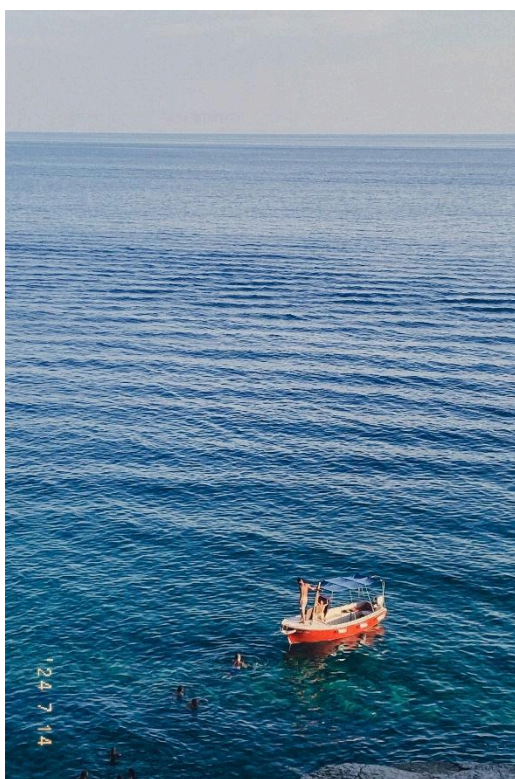
Author



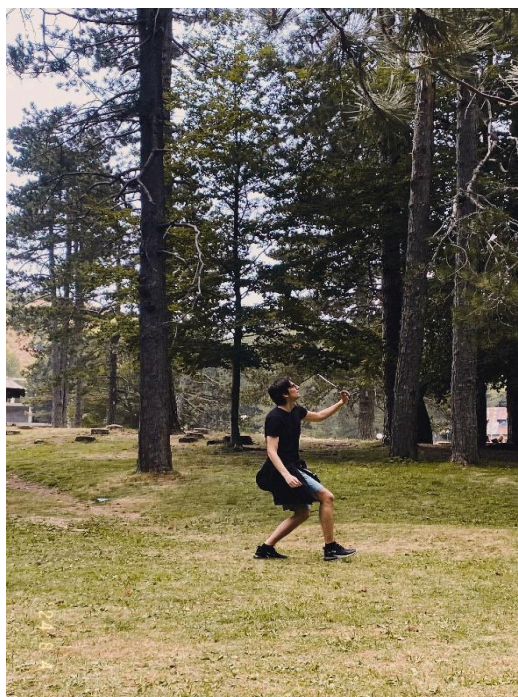
By the water



Beauty of nature



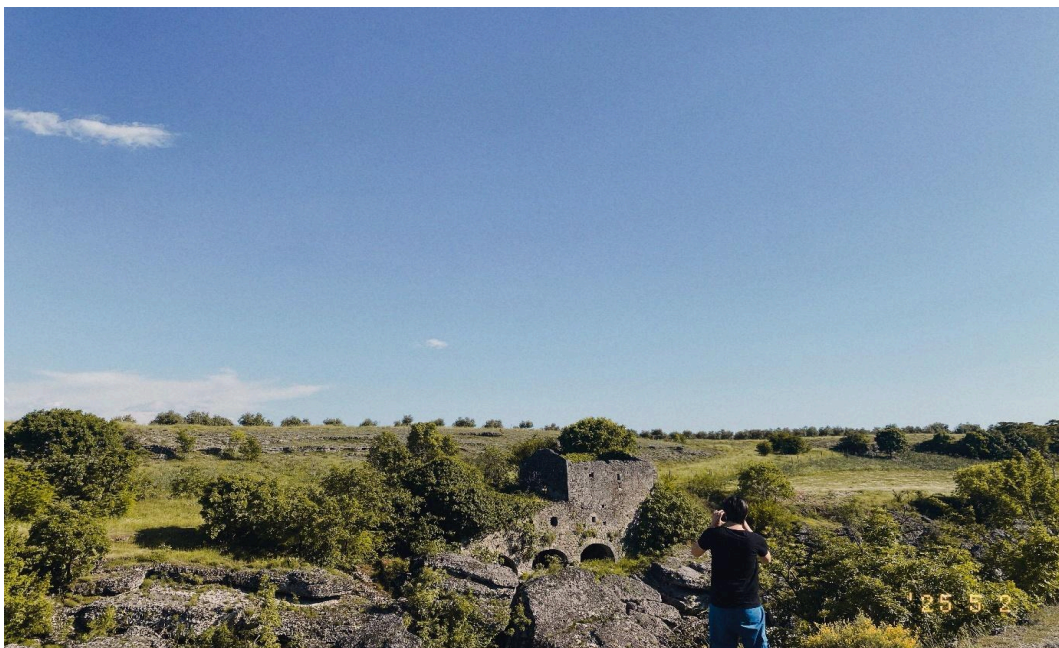
Boat and the sea



Sport



Sleepy



You have to take a picture